

TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE CIUDAD JUÁREZ



Equipo D&F Mindspace:

Delgado Ramírez María Fernanda 23111480

López Cervantes Derek Alejandro 23111586

Carrera: Ingeniería en Sistemas Computacionales

Materia: Ingeniería de Software

Docente: Anita Loya Lozoya

Unidad 4

Documentación técnica del sistema “D&F Mindspace”

Fecha de entrega: 23 de Mayo de 2026.

Índice

1. Datos generales.....	5
2. Control de versiones.....	6
3. Arquitectura del sistema.....	8
4. Requerimientos del sistema.....	9
4.1. Requerimientos de hardware.....	9
4.2. Requerimientos de software.....	9
4.3. Requerimientos adicionales.....	9
5. Instalación del sistema.....	10
5.1. Requisitos previos.....	10
5.2. Obtención del sistema.....	10
6. Estructura del proyecto.....	12
6.1. Descripción de la estructura de las carpetas.....	14
6.1.1. Carpeta css/.....	14
6.1.2. Carpeta js/.....	14
6.1.3. Carpeta php/.....	14
6.1.4. Carpeta uploads/.....	14
6.2. Descripción de la estructura de los archivos.....	15
6.2.1. Archivos de administración (admin_*.php).....	15
6.2.2. Archivos de dashboard (dashboard_*.php / .html).....	15
6.2.3. Archivos de gestión y operaciones.....	15
6.2.4. Archivos principales.....	15
7. Descripción de módulos.....	16
7.1. Módulo de Autenticación.....	16
7.2. Módulo de Gestión de Usuarios.....	16
7.3. Módulo de Administración.....	17
7.4. Módulo de Cursos.....	17
7.5. Módulo de Actividades.....	18
7.6. Módulo de Entregas y Evaluación.....	18
7.7. Módulo de Paneles (Dashboards).....	19
7.8. Módulo de Reportes.....	19
7.9. Módulo de Configuración.....	20
7.10. Módulo de Base de Datos.....	20
7.11. Módulo de API.....	20
8. Base de datos.....	21
8.1. Descripción general.....	21
8.2. Modelo relacional.....	22
Figura Diagrama relacional agrandado.....	22
Figura Diagrama relacional agrandado.....	23
Figura 3.12. Diagrama relacional actualizado de la plataforma D&F	

Minspace.....	24
8.3. Diccionario de datos.....	25
8.3.1. Tabla usuarios.....	25
8.3.2. Tabla cursos.....	26
8.3.3. Tabla actividades.....	27
8.3.4. Tabla entregas.....	28
8.3.5. Tabla evaluaciones.....	28
8.3.6. Tabla inscripciones.....	29
8.3.7. Tabla progreso.....	30
8.3.8. Tabla vinculaciones.....	30
8.3.9. Tablas de administración y monitoreo.....	31
8.4. Scripts SQL de creación.....	32
8.4.1. Tabla usuarios.....	32
8.4.2. Tabla cursos.....	33
8.4.3. Tabla actividades.....	33
8.4.4. Tabla entregas.....	33
8.4.5. Tabla evaluaciones.....	34
8.4.6. Tabla vinculaciones.....	34
8.4.7. Índices implementados.....	35
8.5. Procedimientos almacenados y triggers.....	36
8.5.1. Trigger de actualización automática.....	36
8.6. Consultas representativas del sistema.....	37
8.6.1. Panel Alumno.....	37
8.6.2. Panel Tutor.....	37
8.6.3. Panel Padre.....	38
8.6.4. Panel Administrador.....	39
8.7. Políticas de seguridad y permisos.....	40
9. APIs.....	42
9.1. API de correos electrónicos (SendGrid).....	42
9.1.1. Configuración inicial.....	42
9.1.2. Estructura de implementación.....	42
9.1.3. Integración con los procesos del sistema.....	45
9.1.4. Registro de actividad y monitoreo.....	46
9.1.5. Seguridad y manejo de credenciales.....	46
9.1.6. Limitaciones consideradas.....	46
9.1.7. Resultados obtenidos.....	46
10. Seguridad.....	47
10.1. Autenticación de usuarios.....	47
10.2. Control de acceso por roles.....	47
10.3. Validación de datos.....	48
10.4. Manejo de sesiones.....	48
10.5. Seguridad en la base de datos.....	49

10.6. Manejo de errores.....	49
10.8. Respaldos.....	50
11. Errores.....	51
12. Respaldos.....	52
12.1. Respaldo por servidor completo.....	52
12.2. Ubicación de los respaldos.....	53
12.3. Periodicidad.....	53
13. Mantenimiento.....	55
13.1. Respaldo de la información.....	56
13.2. Restauración del sistema.....	57
13.3. Mantenimiento preventivo y correctivo.....	58
14. Contacto.....	59

1. Datos generales

- **Nombre del sistema:** D&F Mindspace
- **Tipo de sistema:** Aplicación web
- **Versión:** 3.1
- **Fecha de desarrollo:** Enero 2026 - Mayo 2026
- **Autores:** Delgado Ramírez María Fernanda, López Cervantes Derek Alejandro
- **Equipo de Desarrollo:** D&F
- **Institución / Empresa:** D&F Mindspace

2. Control de versiones

El presente apartado tiene como finalidad llevar un registro de las diferentes versiones del sistema D&F Mindspace, permitiendo identificar los cambios realizados, así como los responsables y fechas correspondientes. Esto facilita el mantenimiento, seguimiento y evolución del sistema a lo largo del tiempo.

A continuación, se presenta el historial de versiones:

V	Fecha	Autor(es)	Descripción de cambios
1.0	Enero 2026	López Cervantes Derek Alejandro Delgado Ramírez María Fernanda	Versión inicial del sistema. Incluye funcionalidades básicas como registro de usuarios, inicio de sesión, primeros cursos y primeras actividades. El diseño era simple ya que era principalmente un boceto
1.1	Febrero 2026	López Cervantes Derek Alejandro Delgado Ramírez María Fernanda	Se implementaron validaciones en los formularios de sesiones, se mejoró la interfaz principal y empezamos a diseñar un logotipo. Ya se podían crear cursos y actividades en conjunto pero sin un rol claro.
1.2	Marzo 2026	López Cervantes Derek Alejandro Delgado Ramírez María Fernanda	Se empezaron a agregar funcionalidades, como ejemplo, los roles (solo padre, alumno y tutor), se tenía la idea de agregar vínculos de padres a hijo, además de reestructurar la base de datos mejor. El tutor es el único que puede crear cursos y actividades dentro de los cursos.
2.0	Abril 2026	López Cervantes Derek Alejandro Delgado Ramírez María Fernanda	Se cambió drásticamente el diseño de la página a algo más infantil, con muchos colores en el index, y más equilibrado en los dashboards de cada rol. Se empezó a poner el logotipo de D&F Mindspace, pero solo como un boceto. Además se agregó el rol de administrador para gestionar.
2.1	Abril 2026	López Cervantes Derek Alejandro Delgado Ramírez María Fernanda	El index de nuevo sufrió cambios en el diseño, dejándolo más equilibrado en colores para no saturar demasiado. Además se agregó el logotipo oficial de D&F Mindspace y un sistema de avatares. Se empezó a trabajar en conjunto por medio de Github para más agilidad.

2.2	Mayo 2026	López Cervantes Derek Alejandro Delgado Ramírez María Fernanda	Se empezaron a corregir errores que empezamos a tener en ciertos momentos (mayormente con la base de datos migrada), además de terminar de agregar todas las páginas necesarias para que sea consistente la página (como paginas de detalles, reportes, estadísticas, etc).
2.3	Mayo 2026	López Cervantes Derek Alejandro Delgado Ramírez María Fernanda	Se agregaron más funcionalidades que catalogamos como necesarias, como las misiones vencidas, poder salirte de un curso o los avatares predeterminados, además, se corrigieron errores que comenzamos a descubrir al realizar la matriz de casos de prueba.
3.0	Mayo 2026	López Cervantes Derek Alejandro Delgado Ramírez María Fernanda	Se agregó correctamente la función de vinculación de padres con hijos por medio de códigos de vinculación, también se agregó el API de correos para notificar cuando se envíen o entreguen tareas.
3.1	Mayo 2026	López Cervantes Derek Alejandro Delgado Ramírez María Fernanda	Se corrigieron errores importantes previos a la presentación del sistema.

3. Arquitectura del sistema

El sistema D&F Mindspace está desarrollado bajo una arquitectura de tipo cliente-servidor, la cual permite la interacción entre el usuario final y el sistema a través de un navegador web. Esta arquitectura se compone de una capa de presentación (frontend), una capa lógica (backend) y una capa de datos (base de datos).

En la capa de presentación se utilizan tecnologías como HTML, CSS y JavaScript, las cuales permiten la construcción de interfaces dinámicas e interactivas para el usuario. Asimismo, se emplean recursos externos como tipografías personalizadas mediante Google Fonts y efectos visuales basados en animaciones para mejorar la experiencia de usuario.

La capa lógica está desarrollada en PHP, encargándose de procesar las solicitudes del usuario, gestionar la lógica del sistema, validar datos y establecer la comunicación con la base de datos.

En la capa de datos se utiliza PostgreSQL como sistema gestor de base de datos, implementado en un servidor de máquina virtual, lo que permite una administración robusta, segura y escalable de la información.

El sistema se ejecuta sobre un servidor Apache, utilizando el entorno de desarrollo XAMPP, lo que facilita la implementación y pruebas en un entorno local.

Adicionalmente, para el control de versiones y trabajo colaborativo, se utiliza la plataforma GitHub, permitiendo la gestión eficiente del código fuente y el seguimiento de cambios realizados durante el desarrollo.

Finalmente, el sistema contempla la integración de una API orientada al envío de correos y notificaciones, con el objetivo de mejorar la comunicación con los usuarios y ampliar las funcionalidades del sistema.

4. Requerimientos del sistema

Para el correcto funcionamiento del sistema D&F Mindspace, es necesario cumplir con ciertos requisitos tanto de hardware como de software, los cuales se describen a continuación:

4.1. Requerimientos de hardware

- **Procesador:** Intel Core i3 o equivalente en adelante
- **Memoria RAM:** Mínimo 4 GB
- **Almacenamiento:** Al menos 1 GB de espacio disponible
- **Conectividad:** Acceso a red local o conexión a internet

4.2. Requerimientos de software

- **Sistema operativo:** Windows, Linux o macOS
- **Servidor web:** Apache (incluido en XAMPP)
- **Entorno de ejecución:** XAMPP o equivalente
- **Lenguaje de programación:** PHP 8.x
- **Gestor de base de datos:** PostgreSQL (instalado en servidor de máquina virtual)
- **Navegador web:**
 - Google Chrome (versión 120 o superior)
 - Microsoft Edge (versión 120 o superior)
 - Mozilla Firefox (versión 120 o superior)

4.3. Requerimientos adicionales

- **Acceso a GitHub** para la gestión del código fuente (solo para desarrolladores)
- **Permisos de red** para la conexión con el servidor de base de datos en la máquina virtual
- **Configuración de puertos** habilitados para el funcionamiento de Apache y PostgreSQL

5. Instalación del sistema

A continuación, se describe el procedimiento necesario para la correcta instalación y configuración del sistema D&F Mindspace en un entorno local.

5.1. Requisitos previos

Antes de iniciar la instalación, se debe contar con lo siguiente:

- XAMPP instalado (Apache y PHP)
- PostgreSQL 16 instalado y configurado en una máquina virtual
- Navegador web actualizado
- Acceso al repositorio del sistema en GitHub

5.2. Obtención del sistema

1. Clonar el repositorio con la URL:

```
git clone [https://github.com/bhferx9/D-FMindspace]
```

o descargarlo en formato .zip desde GitHub y descomprimirlo.

2. Copiar la carpeta del proyecto en el directorio:

```
C:\xampp\htdocs\
```

3. Configuración del servidor web

- Abrir el panel de control de XAMPP.
- Iniciar el servicio de **Apache**.
- Verificar que el puerto (por defecto 80) esté disponible.

4. Configuración de la base de datos

- Acceder a PostgreSQL desde la máquina virtual.
- Crear la base de datos:

```
CREATE DATABASE df_mindspace;
```

- Importar el archivo .sql del sistema utilizando una herramienta como pgAdmin o la consola de PostgreSQL.

Nota: Es necesario verificar que la máquina virtual se encuentre en ejecución y que exista conectividad de red entre el servidor local y la base de datos

5. Configuración de la conexión

- Localizar el archivo de configuración config.php
- Editar los siguientes parámetros:

```
$host = "IP de tu red actual";  
$port = "5432";  
$dbname = "df_mindspace";  
$user = "admin_user";  
$password = "admin123";
```

- Guardar los cambios. (CTRL + S)

6. Ejecución del sistema

- Abrir un navegador web.
- Ingresar la siguiente dirección:

http://localhost/D-FMindspace

- Verificar que el sistema cargue correctamente.

*Nota: En caso de que no cargue el sistema, verificar que la carpeta se llame
"D-FMindspace" y se encuentre en el directorio correcto.*

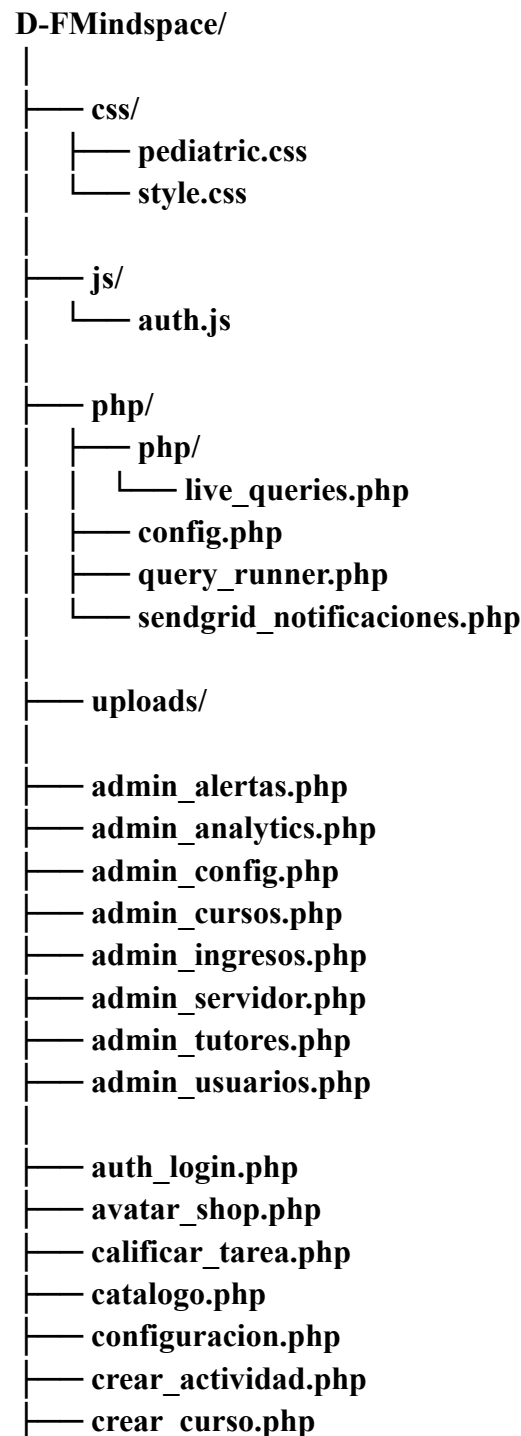
7. Verificación de funcionamiento

- Comprobar que el sistema permite:
 - Acceder al módulo de inicio de sesión
 - Registrar usuario
 - Iniciar sesión con el usuario creado.
- Si es así, entonces la conexión con la base de datos es correcta.

6. Estructura del proyecto

El sistema D&F Mindspace presenta una estructura organizada en carpetas y archivos que permiten la separación de responsabilidades, facilitando su mantenimiento, escalabilidad y comprensión por parte de desarrolladores.

La siguiente estructura representa la organización física de los archivos del sistema al momento de su desarrollo



- dashboard_admin.php
- dashboard_alumno.php
- dashboard_padre.php
- dashboard_padre_detalle.php
- dashboard_tutor.php
- dashboard.html
- detalle_alumno.php
- detalle_entregas.php
- detalles_curso.php
- editar_curso.php
- entrega.php
- notificar_tutor_pendientes.php
- estilo.html
- gestionar_actividades.php
- get_activity_data.php
- guardar_actividad.php
- guardar_entrega.php
- guardar_evaluacion.php
- index.php
- inscribir_logica.php
- login.html
- logout.php
- mis_actividades.php
- mis_cursos.php
- mis_logros.php
- probar_config.php
- procesar_registro.php
- procesar_salir_curso.php
- proceso_registro.php
- prueba.php
- realizar_actividad.php
- registro.php
- reporte_alumnos.php

```
├── reportes.php
├── revisar_entrega.php
├── revisar_entregas.php
├── suscripcion.php
├── test_extensions.php
├── test_pg.php
├── ver_curso.php
├── ver_entrega.php
└── vincular_hijo.php
```

6.1. Descripción de la estructura de las carpetas

6.1.1. Carpeta css/

Contiene los archivos de estilos utilizados para el diseño visual del sistema.

- ***style.css***: estilos generales del sistema
- ***pediatric.css***: estilos específicos para ciertas vistas o módulos

6.1.2. Carpeta js/

Contiene scripts que permiten la interacción del usuario con el sistema.

- ***auth.js***: manejo de autenticación y validaciones del lado del cliente

6.1.3. Carpeta php/

Contiene la lógica principal del sistema desarrollada en PHP.

- ***config.php***: configuración de conexión a la base de datos
- ***query_runner.php***: ejecución de consultas SQL
- ***live_queries.php***: consultas dinámicas o en tiempo real
- ***sendgrid_notificaciones.php***: funcionalidad del API de correos

6.1.4. Carpeta uploads/

Directorio destinado al almacenamiento de archivos subidos por los usuarios (por ejemplo, imágenes o documentos).

6.2. Descripción de la estructura de los archivos

6.2.1. Archivos de administración (admin_*.php)

Contienen funcionalidades exclusivas para administradores, como:

- Gestión de usuarios
- Configuración del sistema
- Administración de cursos, tutores e ingresos

6.2.2. Archivos de dashboard (dashboard_*.php / .html)

Manejan las vistas principales según el tipo de usuario:

- Administrador
- Alumno
- Tutor
- Padre

6.2.3. Archivos de gestión y operaciones

Incluyen funcionalidades específicas del sistema:

- *crear_curso.php, editar_curso.php*: gestión de cursos
- *guardar_*.php*: almacenamiento de información
- *detalle_*.php*: visualización detallada
- *revisar_*.php*: revisión de entregas o actividades
- *notificar_tutor_pendientes.php*: interacción con correos.

6.2.4. Archivos principales

- *index.php*: punto de entrada del sistema
- *login.html*: interfaz de inicio de sesión
- *dashboard.html*: estructura base del panel principal

La estructura del proyecto sigue una organización modular basada en funcionalidades, aunque no se implementa un framework específico, lo que permite flexibilidad en el desarrollo

7. Descripción de módulos

El sistema D&F Mindspace está compuesto por diversos módulos funcionales que permiten la gestión integral de usuarios, cursos, actividades y procesos internos. A continuación, se describen los principales módulos del sistema:

7.1. Módulo de Autenticación

Encargado de gestionar el acceso al sistema mediante credenciales de usuario.

Funciones principales:

- Inicio de sesión
- Validación de credenciales
- Cierre de sesión
- Control de sesiones por tipo de usuario

Archivos relacionados:

- *login.html*
- *auth_login.php*
- *logout.php*
- *auth.js*

7.2. Módulo de Gestión de Usuarios

Permite la administración de los usuarios dentro del sistema según su rol.

Funciones principales:

- Registro de usuarios
- Edición de datos
- Visualización de información
- Vinculación de cuentas (por ejemplo, padres e hijos)

Archivos relacionados:

- *registro.php*
- *procesar_registro.php*
- *admin_usuarios.php*
- *vincular_hijo.php*

7.3. Módulo de Administración

Contiene funcionalidades exclusivas para el administrador del sistema.

Funciones principales:

- Gestión de cursos
- Administración de tutores
- Configuración del sistema
- Visualización de reportes e indicadores

Archivos relacionados:

- *admin_cursos.php*
- *admin_tutores.php*
- *admin_config.php*
- *admin_analytics.php*
- *admin_ingresos.php*
- *admin_servidor.php*

7.4. Módulo de Cursos

Permite la creación, edición y visualización de cursos dentro del sistema.

Funciones principales:

- Creación de cursos
- Edición de cursos
- Inscripción a cursos
- Visualización de catálogo

Archivos relacionados:

- *crear_curso.php*
- *editar_curso.php*
- *catalogo.php*
- *inscribir_logica.php*
- *ver_curso.php*

7.5. Módulo de Actividades

Gestiona las actividades asignadas dentro de los cursos.

Funciones principales:

- Creación de actividades
- Consulta de actividades
- Entrega de actividades
- Evaluación de tareas

Archivos relacionados:

- *crear_actividad.php*
- *gestionar_actividades.php*
- *realizar_actividad.php*
- *guardar_actividad.php*
- *guardar_entrega.php*
- *calificar_tarea.php*

7.6. Módulo de Entregas y Evaluación

Permite el seguimiento y revisión de actividades entregadas por los usuarios.

Funciones principales:

- Revisión de entregas
- Visualización de detalles
- Evaluación de desempeño

Archivos relacionados:

- *revisar_entregas.php*
- *revisar_entrega.php*
- *detalle_entregas.php*
- *ver_entrega.php*
- *guardar_evaluacion.php*

7.7. Módulo de Paneles (Dashboards)

Proporciona interfaces personalizadas según el tipo de usuario.

Funciones principales:

- Visualización de información relevante
- Acceso a funcionalidades según rol

Tipos de dashboards:

- Administrador
- Alumno
- Tutor
- Padre

Archivos relacionados:

- *dashboard_admin.php*
- *dashboard_alumno.php*
- *dashboard_tutor.php*
- *dashboard_padre.php*
- *dashboard_padre_detalle.php*

7.8. Módulo de Reportes

Permite la generación y visualización de reportes del sistema.

Funciones principales:

- Reportes de alumnos
- Reportes generales

Archivos relacionados:

- *reporte_alumnos.php*
- *reportes.php*

7.9. Módulo de Configuración

Permite ajustar parámetros del sistema y configuraciones del usuario.

Funciones principales:

- Configuración de cuenta
- Ajustes generales

Archivos relacionados:

- *configuracion.php*
- *probar_config.php*

7.10. Módulo de Base de Datos

Gestiona la conexión y ejecución de consultas hacia la base de datos.

Funciones principales:

- Conexión a PostgreSQL
- Ejecución de consultas
- Manejo de datos
- Api de correos

Archivos relacionados:

- *config.php*
- *query_runner.php*
- *live_queries.php*

7.11. Módulo de API

Gestiona las notificaciones en los correos registrados que sean reales.

Funciones principales:

- Manejo de datos
- Api de correos

Archivos relacionados:

- *sendgrid_notificaciones.php*
- *notificar_tutor_pendientes.php*

8. Base de datos

8.1. Descripción general

El sistema D&F Mindspace utiliza PostgreSQL 16 como sistema gestor de bases de datos, implementado en una máquina virtual independiente para garantizar la disponibilidad, seguridad y escalabilidad de la información.

La base de datos, denominada df_mindspace, sigue un modelo relacional normalizado en Tercera Forma Normal (3FN), lo que asegura la integridad referencial y minimiza la redundancia de datos. Está compuesta por 17 tablas principales que abarcan desde la gestión de usuarios con sus respectivos roles (alumno, tutor, padre, administrador) hasta el seguimiento detallado de cursos, actividades, entregas, evaluaciones, progreso académico y vinculaciones familiares.

Propietarios de los objetos:

Propietario	Objetos gestionados
df_admin	Usuarios, cursos, actividades, entregas, evaluaciones, inscripciones, progreso, notificaciones
admin_user	Alertas de administrador, configuración del sistema, notificaciones configuradas
postgres	Respuestas de examen, vinculaciones padre-hijo

Características técnicas:

- Codificación: UTF-8
- Extensión habilitada: pg_stat_statements para monitoreo de rendimiento
- Índices implementados: 9 índices estratégicos para optimización de consultas
- Triggers: 1 trigger para actualización automática de fechas

8.2. Modelo relacional

El siguiente diagrama relacional representa la estructura lógica de la base de datos, mostrando las entidades principales, sus atributos y las relaciones entre ellas:

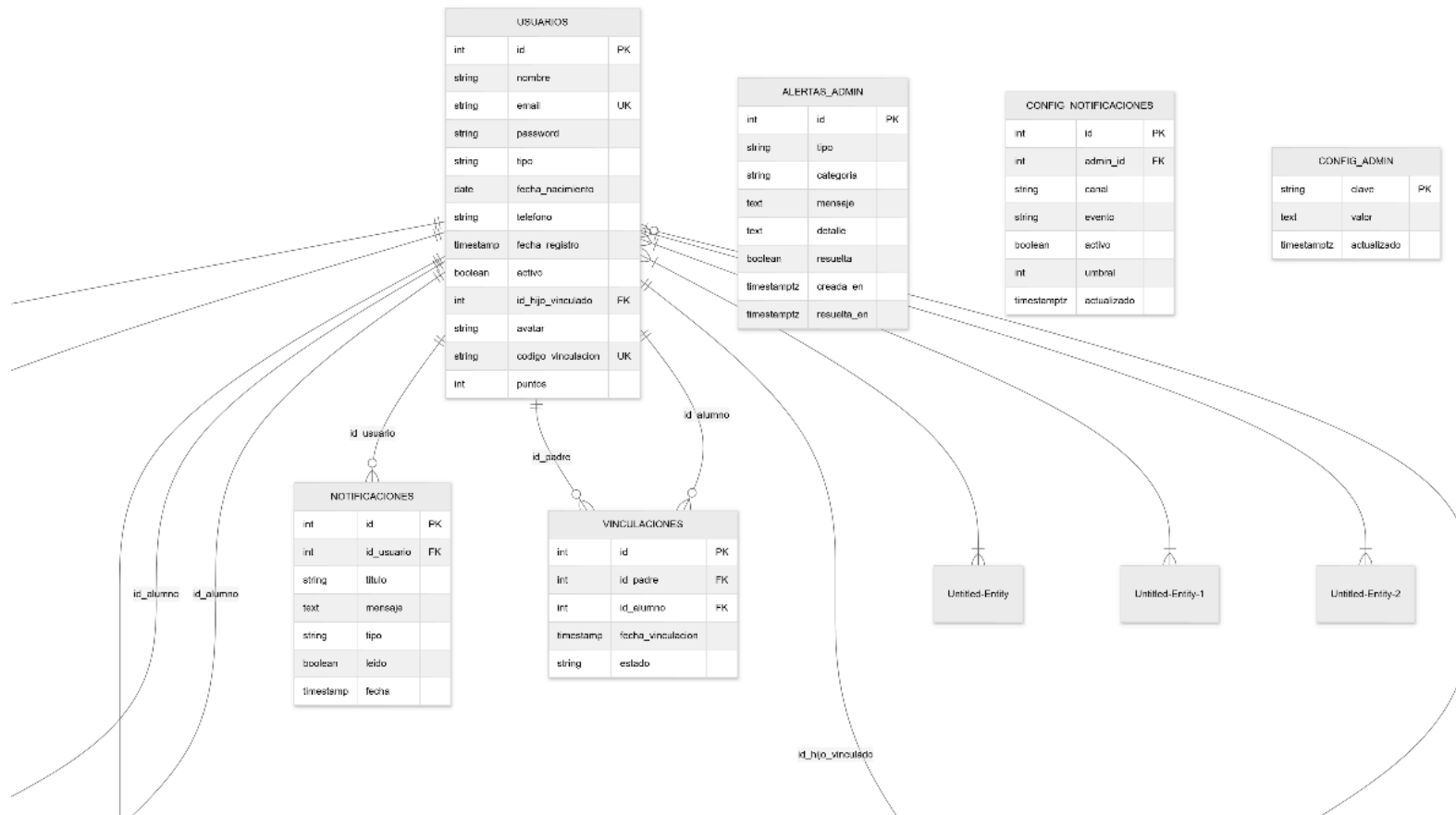


Figura Diagrama relacional agrandado

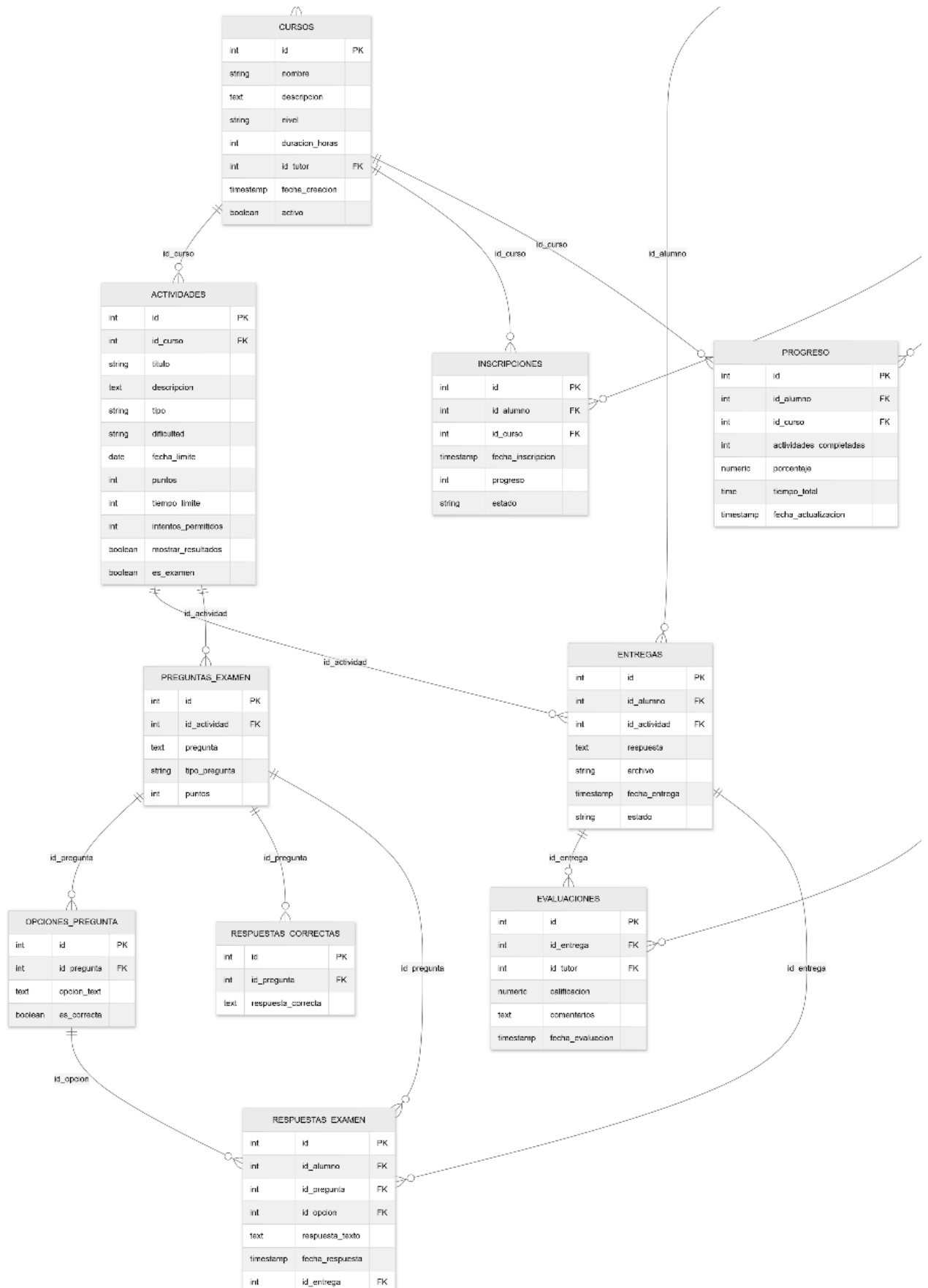
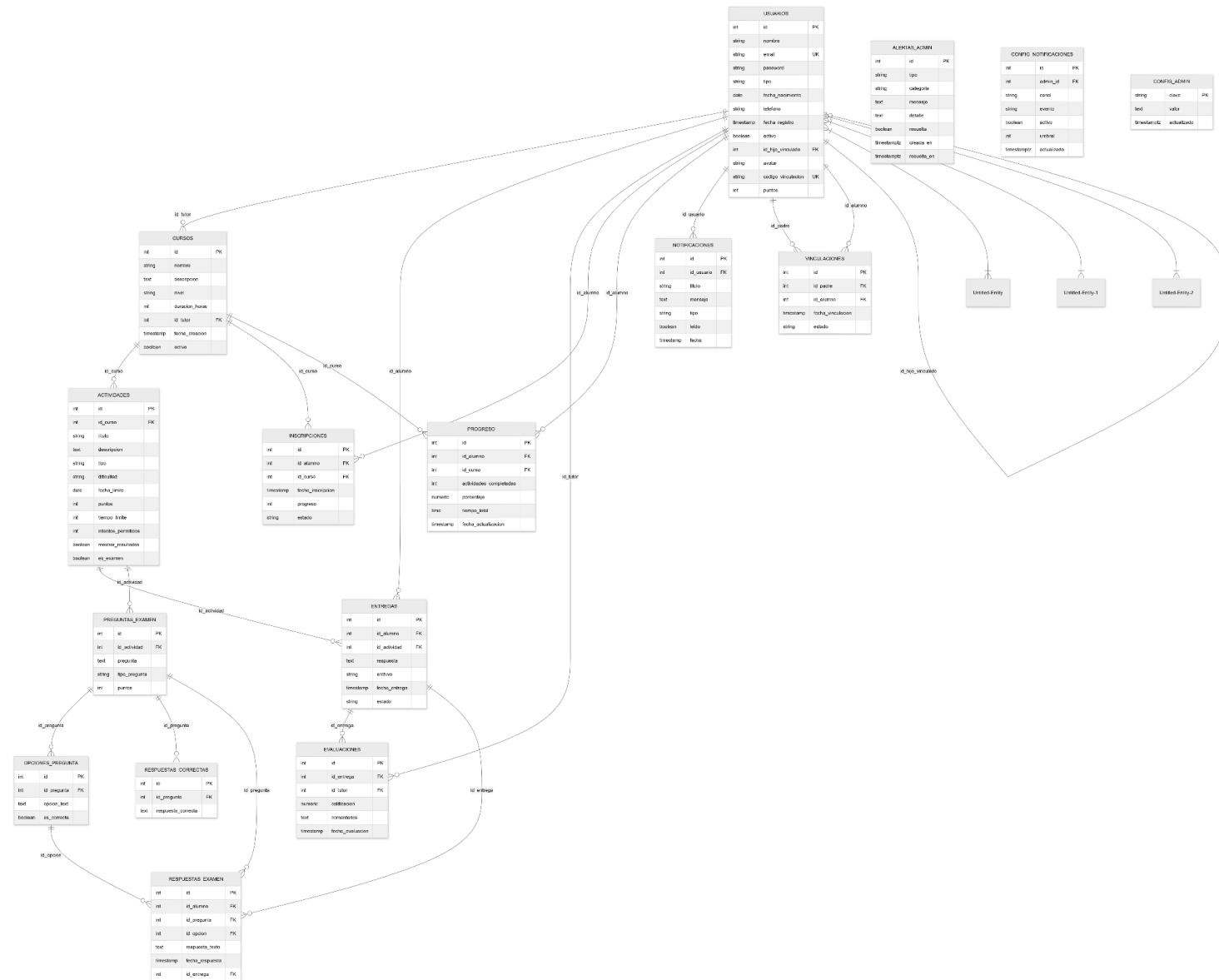


Figura Diagrama relacional agrandado

Figura 3.12. Diagrama relacional actualizado de la plataforma D&F Mindspace



8.3. Diccionario de datos

A continuación se describe la estructura de cada tabla, incluyendo el nombre del campo, tipo de dato, restricciones y una breve descripción de su propósito.

8.3.1. Tabla usuarios

Campo	Tipo	Restricción	Descripción
id	SERIAL	PK	Identificador único del usuario
nombre	VARCHAR(100)	NOT NULL	Nombre completo del usuario
email	VARCHAR(100)	UNIQUE, NOT NULL	Correo electrónico (usado para login)
password	VARCHAR(255)	NOT NULL	Hash de la contraseña (bcrypt)
tipo	VARCHAR(30)	CHECK (alumno, tutor, padre, admin)	Rol del usuario en el sistema
fecha_nacimiento	DATE	-	Fecha de nacimiento (para calcular edad)
telefono	VARCHAR(20)	-	Número de contacto
fecha_registro	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha de creación de la cuenta
activo	BOOLEAN	DEFAULT true	Indica si la cuenta está activa

id_hijo_vinculado	INT	FK (usuarios.id)	Para vinculación directa padre→hijo
avatar	VARCHAR(50)	DEFAULT 'panda'	Nombre del avatar del usuario
codigo_vinculacion	VARCHAR(50)	UNIQUE	Código para vincular padre con hijo
puntos	INT	DEFAULT 0	Puntos acumulados por el usuario

8.3.2. Tabla cursos

Campo	Tipo	Restricción	Descripción
id	SERIAL	PK	Identificador único del curso
nombre	VARCHAR(200))	NOT NULL	Título del curso
descripcion	TEXT	-	Descripción detallada del curso
nivel	VARCHAR(50)	-	Básico, Intermedio, Avanzado
duracion_hora s	INT	-	Duración estimada en horas
id_tutor	INT	FK (usuarios.id)	Tutor que creó/administra el curso
fecha_creacio n	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha de creación del curso

activo	BOOLEAN	DEFAULT true	Indica si el curso está disponible
--------	---------	--------------	------------------------------------

8.3.3. Tabla actividades

Campo	Tipo	Restricción	Descripción
id	SERIAL	PK	Identificador único de la actividad
id_curso	INT	FK (cursos.id)	Curso al que pertenece
titulo	VARCHAR(200)	NOT NULL	Título de la actividad
descripcion	TEXT	-	Instrucciones o descripción
tipo	VARCHAR(50)	-	Tarea, Quiz, Examen, etc.
dificultad	VARCHAR(50)	-	Fácil, Normal, Difícil
fecha_limite	DATE	-	Fecha límite de entrega
puntos	INT	-	Puntos máximos a obtener
tiempo_limite	INT	DEFAULT 60	Tiempo límite en minutos
intentos_permitidos	INT	DEFAULT 1	Intentos permitidos
mostrar_resultados	BOOLEAN	DEFAULT true	Muestra resultados al alumno

es_examen	BOOLEAN	DEFAULT false	Indica si es tipo examen
-----------	---------	------------------	--------------------------

8.3.4. Tabla entregas

Campo	Tipo	Restricción	Descripción
id	SERIAL	PK	Identificador único de la entrega
id_alumno	INT	FK (usuarios.id)	Alumno que entrega
id_actividad	INT	FK (actividades.id)	Actividad entregada
respuesta	TEXT	-	Respuesta de texto del alumno
archivo	VARCHAR(255)	-	Ruta del archivo adjunto
fecha_entrega	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Momento de la entrega
estado	VARCHAR(50)	DEFAULT 'pendiente'	pendiente, calificado, atrasado

8.3.5. Tabla evaluaciones

Campo	Tipo	Restricción	Descripción
id	SERIAL	PK	Identificador único
id_entrega	INT	FK (entregas.id)	Entrega evaluada

id_tutor	INT	FK (usuarios.id)	Tutor que califica
calificacion	NUMERIC(5, 2)	-	Nota obtenida (0-100)
comentarios	TEXT	-	Retroalimentación del tutor
fecha_evaluacion	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha de la calificación

8.3.6. Tabla inscripciones

Campo	Tipo	Restricción	Descripción
id	SERIAL	PK	Identificador único
id_alumno	INT	FK (usuarios.id)	Alumno inscrito
id_curso	INT	FK (cursos.id)	Curso al que se inscribe
fecha_inscripcion	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha de inscripción
progreso	INT	DEFAULT 0	Porcentaje de progreso (0-100)
estado	VARCHAR(50)	DEFAULT 'activo'	activo, completado, abandonado

8.3.7. Tabla progreso

Campo	Tipo	Restricción	Descripción
id	SERIAL	PK	Identificador único
id_alumno	INT	FK (usuarios.id)	Alumno
id_curso	INT	FK (cursos.id)	Curso
actividades_completadas	INT	DEFAULT 0	Número de actividades finalizadas
porcentaje	NUMERIC(5, 2)	DEFAULT 0	Porcentaje de avance
tiempo_total	TIME	-	Tiempo total dedicado
fecha_actualizacion	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Última actualización

8.3.8. Tabla vinculaciones

Campo	Tipo	Restricción	Descripción
id	SERIAL	PK	Identificador único
id_padre	INT	FK (usuarios.id)	Padre/familiar
id_alumno	INT	FK (usuarios.id)	Hijo/alumno vinculado
fecha_vinculacion	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Fecha de la vinculación

estado	VARCHAR(10)	CHECK (activo, pendiente, rechazado)	Estado de la relación
--------	-------------	---	-----------------------

8.3.9. Tablas de administración y monitoreo

Tabla	Descripción
preguntas_examen	Almacena las preguntas de actividades tipo examen
opciones_pregunta	Opciones de respuesta para preguntas de opción múltiple
respuestas_correctas	Respuestas esperadas para preguntas abiertas
respuestas_examen	Respuestas de los alumnos a exámenes
notificaciones	Notificaciones del sistema para usuarios
alertas_admin	Alertas del sistema para administradores
config_admin	Configuraciones globales del sistema
config_notificaciones	Preferencias de notificaciones por administrador

8.4. Scripts SQL de creación

A continuación se presentan los scripts SQL para la creación de las tablas principales del sistema.

8.4.1. Tabla *usuarios*

```
CREATE TABLE public.usuarios (  
    id integer NOT NULL,  
    nombre character varying(100) NOT NULL,  
    email character varying(100) NOT NULL,  
    password character varying(255) NOT NULL,  
    tipo character varying(30) NOT NULL,  
    fecha_nacimiento date,  
    telefono character varying(20),  
    fecha_registro timestamp without time zone DEFAULT  
CURRENT_TIMESTAMP NOT NULL,  
    activo boolean DEFAULT true,  
    id_hijo_vinculado integer,  
    avatar character varying(50) DEFAULT 'panda'::character varying,  
    codigo_vinculacion character varying(50),  
    puntos integer DEFAULT 0,  
    CONSTRAINT usuarios_tipo_check CHECK (  
        ((tipo)::text = ANY (  
            (ARRAY[  
                'alumno'::character varying,  
                'tutor'::character varying,  
                'padre'::character varying,  
                'admin'::character varying  
            ]::text[]  
        ))  
    )  
);  
  
ALTER TABLE public.usuarios  
    ADD CONSTRAINT usuarios_pkey PRIMARY KEY (id),  
    ADD CONSTRAINT usuarios_email_key UNIQUE (email),  
    ADD CONSTRAINT usuarios_codigo_vinculacion_key UNIQUE  
(codigo_vinculacion),  
    ADD CONSTRAINT usuarios_id_hijo_vinculado_fkey  
        FOREIGN KEY (id_hijo_vinculado)  
        REFERENCES public.usuarios(id);
```


8.4.2. Tabla cursos

```
CREATE TABLE public.cursos (  
    id integer NOT NULL,  
    nombre character varying(200) NOT NULL,  
    descripcion text,  
    nivel character varying(50),  
    duracion_horas integer,  
    id_tutor integer,  
    fecha_creacion timestamp without time zone DEFAULT  
CURRENT_TIMESTAMP NOT NULL,  
    activo boolean DEFAULT true  
);
```

```
ALTER TABLE public.cursos  
    ADD CONSTRAINT cursos_pkey PRIMARY KEY (id),  
    ADD CONSTRAINT cursos_id_tutor_fkey  
        FOREIGN KEY (id_tutor)  
        REFERENCES public.usuarios(id);
```

8.4.3. Tabla actividades

```
CREATE TABLE public.actividades (  
    id integer NOT NULL,  
    id_curso integer,  
    titulo character varying(200) NOT NULL,  
    descripcion text,  
    tipo character varying(50),  
    dificultad character varying(50),  
    fecha_limite date,  
    puntos integer,  
    tiempo_limite integer DEFAULT 60,  
    intentos_permitidos integer DEFAULT 1,  
    mostrar_resultados boolean DEFAULT true,  
    es_examen boolean DEFAULT false  
);
```

```
ALTER TABLE public.actividades  
    ADD CONSTRAINT actividades_pkey PRIMARY KEY (id),  
    ADD CONSTRAINT actividades_id_curso_fkey  
        FOREIGN KEY (id_curso)  
        REFERENCES public.cursos(id);
```

8.4.4. Tabla entregas

```
CREATE TABLE public.entregas (  
    id integer NOT NULL,  
    id_alumno integer,  
    id_actividad integer,
```

```

        respuesta text,
        archivo character varying(255),
        fecha_entrega timestamp without time zone DEFAULT
CURRENT_TIMESTAMP NOT NULL,
        estado character varying(50) DEFAULT 'pendiente'::character
varying
);

```

```

ALTER TABLE public.entregas
    ADD CONSTRAINT entregas_pkey PRIMARY KEY (id),
    ADD CONSTRAINT entregas_id_alumno_fkey
        FOREIGN KEY (id_alumno)
        REFERENCES public.usuarios(id),
    ADD CONSTRAINT entregas_id_actividad_fkey
        FOREIGN KEY (id_actividad)
        REFERENCES public.actividades(id);

```

8.4.5. Tabla *evaluaciones*

```

CREATE TABLE public.evaluaciones (
    id integer NOT NULL,
    id_entrega integer,
    id_tutor integer,
    calificacion numeric(5,2),
    comentarios text,
    fecha_evaluacion timestamp without time zone DEFAULT
CURRENT_TIMESTAMP NOT NULL
);

```

```

ALTER TABLE public.evaluaciones
    ADD CONSTRAINT evaluaciones_pkey PRIMARY KEY (id),
    ADD CONSTRAINT evaluaciones_id_entrega_fkey
        FOREIGN KEY (id_entrega)
        REFERENCES public.entregas(id),
    ADD CONSTRAINT evaluaciones_id_tutor_fkey
        FOREIGN KEY (id_tutor)
        REFERENCES public.usuarios(id);

```

8.4.6. Tabla *vinculaciones*

```

CREATE TABLE public.vinculaciones (
    id integer NOT NULL,
    id_padre integer NOT NULL,
    id_alumno integer NOT NULL,
    fecha_vinculacion timestamp without time zone DEFAULT
CURRENT_TIMESTAMP NOT NULL,

```

```

estado character varying(10) DEFAULT 'activo'::character varying,

CONSTRAINT vinculaciones_estado_check CHECK (
    ((estado)::text = ANY (
        (ARRAY[
            'activo'::character varying,
            'pendiente'::character varying,
            'rechazado'::character varying
        ])::text[]
    ))
)
);

ALTER TABLE public.vinculaciones
    ADD CONSTRAINT vinculaciones_pkey PRIMARY KEY (id),
    ADD CONSTRAINT vinculaciones_id_padre_id_alumno_key
        UNIQUE (id_padre, id_alumno),

    ADD CONSTRAINT vinculaciones_id_alumno_fkey
        FOREIGN KEY (id_alumno)
        REFERENCES public.usuarios(id)
        ON DELETE CASCADE,

    ADD CONSTRAINT vinculaciones_id_padre_fkey
        FOREIGN KEY (id_padre)
        REFERENCES public.usuarios(id)
        ON DELETE CASCADE;

```

8.4.7. Índices implementados

```

-- Optimización de consultas frecuentes

CREATE INDEX idx_inscripciones_alumno_curso
    ON public.inscripciones USING btree (id_alumno, id_curso);

CREATE INDEX idx_progreso_alumno_curso
    ON public.progreso USING btree (id_alumno, id_curso);

CREATE INDEX idx_evaluaciones_entrega
    ON public.evaluaciones USING btree (id_entrega);

CREATE INDEX idx_evaluaciones_tutor
    ON public.evaluaciones USING btree (id_tutor);

CREATE INDEX idx_preguntas_actividad
    ON public.preguntas_examen USING btree (id_actividad);

```

```

CREATE INDEX idx_usuarios_tipo
    ON public.usuarios USING btree (tipo);

CREATE INDEX idx_usuarios_activo
    ON public.usuarios USING btree (activo);

CREATE INDEX idx_respuestas_alumno_pregunta
    ON public.respuestas_examen USING btree (id_alumno, id_pregunta);

```

8.5. Procedimientos almacenados y triggers

8.5.1. Trigger de actualización automática

El sistema implementa un trigger que actualiza automáticamente el campo `fecha_actualizacion` cada vez que se modifica un registro en la tabla `progreso`.

```

-- Función del trigger

CREATE FUNCTION public.update_fecha_actualizacion()
RETURNS TRIGGER
LANGUAGE plpgsql AS $$

BEGIN
    NEW.fecha_actualizacion = CURRENT_TIMESTAMP;
    RETURN NEW;
END;

$$;

-- Trigger asociado a la tabla progreso

CREATE TRIGGER trigger_update_progreso
    BEFORE UPDATE ON public.progreso
    FOR EACH ROW
    EXECUTE FUNCTION public.update_fecha_actualizacion();

```

Propósito: Mantener actualizada la marca de tiempo cada vez que se modifica el progreso de un alumno en un curso, sin necesidad de que la aplicación lo maneje explícitamente.

8.6. Consultas representativas del sistema

A continuación se presentan consultas SQL representativas utilizadas en los diferentes módulos del sistema, organizadas por panel.

8.6.1. Panel Alumno

Obtener cursos activos del alumno

```
SELECT c.nombre, c.descripcion, i.progreso, i.id_curso, c.nivel
FROM inscripciones i
JOIN cursos c ON i.id_curso = c.id
WHERE i.id_alumno = ? AND i.estado = 'activo';
```

Obtener misiones recientes (últimas 4 entregas)

```
SELECT a.titulo, e.fecha_entrega, e.estado, ev.calificacion
FROM entregas e
JOIN actividades a ON e.id_actividad = a.id
LEFT JOIN evaluaciones ev ON e.id = ev.id_entrega
WHERE e.id_alumno = ?
ORDER BY e.fecha_entrega DESC
LIMIT 4;
```

Obtener avatar del alumno (con valor predeterminado)

```
SELECT COALESCE/avatar, 'panda') AS avatar
FROM usuarios
WHERE id = ?;
```

Contar actividades calificadas (para desbloquear avatares)

```
SELECT COUNT(DISTINCT e.id_actividad) AS total
FROM entregas e
WHERE e.id_alumno = ? AND e.estado = 'calificado';
```

8.6.2. Panel Tutor

Contar entregas pendientes de revisión

```
SELECT COUNT(*) AS total
FROM entregas e
JOIN actividades a ON e.id_actividad = a.id
JOIN cursos c ON a.id_curso = c.id
WHERE c.id_tutor = ? AND e.estado = 'pendiente';
```

Obtener cursos del tutor con número de inscritos

```
SELECT c.*, COUNT(i.id) AS inscritos
FROM cursos c
LEFT JOIN inscripciones i ON c.id = i.id_curso
WHERE c.id_tutor = ?
GROUP BY c.id
ORDER BY c.fecha_creacion DESC;
```

Obtener entregas pendientes con priorización por antigüedad

```
SELECT
    e.id AS entrega_id,
    u.nombre AS alumno_nombre,
    a.titulo AS actividad_nombre,
    c.nombre AS curso_nombre,
    e.fecha_entrega,
    ev.calificacion,
    EXTRACT(EPOCH FROM (CURRENT_TIMESTAMP - e.fecha_entrega))/3600 AS
horas_pasadas
FROM entregas e
JOIN usuarios u ON e.id_alumno = u.id
JOIN actividades a ON e.id_actividad = a.id
JOIN cursos c ON a.id_curso = c.id
LEFT JOIN evaluaciones ev ON e.id = ev.id_entrega
WHERE c.id_tutor = ? AND e.estado = 'pendiente'
ORDER BY horas_pasadas DESC;
```

8.6.3. Panel Padre

Obtener hijos vinculados al padre con estadísticas

```
SELECT
    u.id,
    u.nombre,
    u.avatar,
    EXTRACT(YEAR FROM age(CURRENT_DATE, u.fecha_nacimiento)) AS edad,

    (
        SELECT COUNT(*)
        FROM inscripciones
        WHERE id_alumno = u.id AND estado = 'activo'
    ) AS cursos_activos,

    (
        SELECT COALESCE(SUM(actividades_completadas), 0)
        FROM progreso
```

```

        WHERE id_alumno = u.id
    ) AS completadas
FROM vinculaciones v
JOIN usuarios u ON v.id_alumno = u.id
WHERE v.id_padre = ?
    AND v.estado = 'activo'
    AND u.activo = TRUE
ORDER BY u.nombre;

```

Calcular promedio de calificaciones del hijo

```

SELECT COALESCE(AVG(ev.calificacion), 0) AS promedio
FROM evaluaciones ev
JOIN entregas en ON ev.id_entrega = en.id
WHERE en.id_alumno = ?;

```

8.6.4. Panel Administrador

Distribución de usuarios por tipo

```

SELECT tipo, COUNT(*) AS total
FROM usuarios
GROUP BY tipo;

```

Top 5 cursos con más inscripciones

```

SELECT c.nombre, COUNT(i.id) AS inscripciones
FROM cursos c
JOIN inscripciones i ON c.id = i.id_curso
WHERE i.estado = 'activo'
GROUP BY c.id, c.nombre
ORDER BY inscripciones DESC
LIMIT 5;

```

Top 5 cursos con mejor promedio de calificaciones

```

SELECT
    c.nombre,
    AVG(ev.calificacion) AS promedio,
    COUNT(ev.id) AS evaluaciones
FROM cursos c
JOIN actividades a ON a.id_curso = c.id
JOIN entregas en ON en.id_actividad = a.id
JOIN evaluaciones ev ON ev.id_entrega = en.id
GROUP BY c.id, c.nombre
HAVING COUNT(ev.id) > 0
ORDER BY evaluaciones DESC
LIMIT 5;

```

8.7. Políticas de seguridad y permisos

El sistema implementa un esquema de permisos diferenciados por rol de usuario a nivel de base de datos, garantizando que cada tipo de usuario tenga acceso únicamente a la información que le corresponde.

Tabla	alumno_user	tutor_user	padre_user	admin_user
usuarios	-	-	-	ALL
cursos	SELECT	SELECT, UPDATE	SELECT	ALL
actividades	SELECT	SELECT, UPDATE	-	ALL
entregas	SELECT	SELECT, UPDATE	-	ALL
evaluaciones	SELECT	SELECT, UPDATE	-	ALL
inscripciones	-	-	-	ALL
progreso	SELECT	SELECT, UPDATE	SELECT	ALL
notificaciones	-	-	SELECT	ALL
vinculaciones	-	-	-	ALL

Ejemplo de asignación de permisos

-- Permisos para tabla actividades

```
GRANT SELECT
ON TABLE public.actividades
TO alumno_user;
```

```
GRANT SELECT, UPDATE
ON TABLE public.actividades
TO tutor_user;
```

```
GRANT ALL
ON TABLE public.actividades
TO admin_user;
```

-- Permisos para tabla cursos

```
GRANT SELECT
ON TABLE public.cursos
TO alumno_user;
```



```
GRANT SELECT, UPDATE
ON TABLE public.cursos
TO tutor_user;
```

```
GRANT SELECT
ON TABLE public.cursos
TO padre_user;
```

```
GRANT ALL
ON TABLE public.cursos
TO admin_user;
```

Consideraciones de seguridad adicionales

- Las contraseñas se almacenan utilizando el algoritmo bcrypt mediante password_hash() en PHP.
- Los correos electrónicos tienen restricción UNIQUE para evitar duplicidad.
- Las relaciones padre-hijo están protegidas por la tabla vinculaciones, que valida el estado activo.
- Las claves foráneas utilizan ON DELETE CASCADE donde es seguro (respuestas, opciones) y SET NULL en otros casos.

Este apartado documenta de manera completa la base de datos del sistema D&F Mindspace, proporcionando la información necesaria para su mantenimiento, evolución y administración.

9. APIs

El sistema D&F Mindspace incorpora el uso de una API externa para la gestión de notificaciones por correo electrónico, lo que permite mantener informados a los usuarios sobre eventos relevantes dentro de la plataforma.

9.1. API de correos electrónicos (SendGrid)

Con el objetivo de automatizar el envío de notificaciones y mejorar la comunicación entre los diferentes actores del sistema (alumnos, tutores y padres), se integró la API de SendGrid, un servicio especializado en la entrega de correos electrónicos a través de Internet. Esta API permite que la plataforma envíe mensajes de forma programática sin necesidad de configurar un servidor de correo tradicional, lo que simplifica la infraestructura y aumenta la confiabilidad en la entrega de los mensajes.

9.1.1. Configuración inicial

El proceso de configuración de la API de SendGrid comenzó con la creación de una cuenta en la plataforma oficial de SendGrid. Una vez verificada la cuenta, se procedió a generar una clave de API (API Key), la cual funciona como un token de autenticación que permite a la aplicación web comunicarse de manera segura con los servidores de SendGrid. Adicionalmente, se verificó la dirección de correo electrónico mariferdelgadoim@gmail.com como remitente autorizado, lo que garantiza que los correos enviados por el sistema sean aceptados por los servidores de los destinatarios.

Por razones de seguridad, esta API Key no se incluye en el repositorio público de GitHub. En su lugar, se configuró un archivo .gitignore que excluye el archivo que contiene las credenciales del control de versiones, evitando así que información sensible quede expuesta.

9.1.2. Estructura de implementación

La integración de la API de SendGrid se realizó mediante la creación de un archivo centralizado denominado php/sendgrid_notificaciones.php, el cual contiene toda la lógica necesaria para construir y enviar los distintos tipos de correos que requiere el sistema. Este archivo se organiza en varias funciones especializadas, cada una de las cuales atiende un caso de uso específico.

Función base de envío

La función base sendgrid_email() es la encargada de establecer la comunicación con la API de SendGrid. Esta función recibe como parámetros el correo del destinatario, su nombre, el asunto del mensaje y el contenido en formato HTML. Internamente, construye una petición HTTP de tipo POST con un cuerpo en formato JSON, tal como lo requiere la API de SendGrid, y la envía utilizando la extensión cURL de PHP. El código de esta función es el siguiente:

```

function sendgrid_email(string $to_email, string $to_name, string
$subject, string $html_body): bool {
    $payload = json_encode([
        'personalizations' => [[
            'to' => [['email' => $to_email, 'name' => $to_name]]
        ]],
        'from' => [
            'email' => SENDGRID_FROM_EMAIL,
            'name' => SENDGRID_FROM_NAME
        ],
        'subject' => $subject,
        'content' => [[
            'type' => 'text/html',
            'value' => $html_body
        ]]
    ]);
    $ch = curl_init('https://api.sendgrid.com/v3/mail/send');
    curl_setopt_array($ch, [
        CURLOPT_POST => true,
        CURLOPT_POSTFIELDS => $payload,
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_HTTPHEADER => [
            'Authorization: Bearer ' . SENDGRID_API_KEY,
            'Content-Type: application/json'
        ]
    ]);
    $response = curl_exec($ch);
    $http_code = curl_getinfo($ch, CURLINFO_HTTP_CODE);
    curl_close($ch);
    return $http_code === 202;
}

```

La API de SendGrid responde con un código HTTP 202 cuando el correo ha sido aceptado para su envío, por lo que la función retorna true en ese caso y false en cualquier otro, lo que permite al sistema conocer el resultado de la operación.

Funciones de notificación especializadas

A partir de la función base, se construyeron funciones específicas que representan cada tipo de notificación que el sistema puede generar. Estas funciones se encargan de construir el contenido HTML del mensaje, adaptándolo al contexto de cada evento.

Notificación al tutor por nueva entrega

La función `notificar_tutor_nueva_entrega()` se ejecuta automáticamente en el momento en que un alumno realiza la entrega de una actividad. Esta función envía un correo al tutor del curso, informándole que tiene una nueva entrega pendiente de revisión. El mensaje incluye detalles como el nombre del alumno, el título de la actividad y el curso al que pertenece, además de un enlace directo para acceder a la revisión.

```
function notificar_tutor_nueva_entrega(
    string $tutor_email,
    string $tutor_nombre,
    string $alumno_nombre,
    string $actividad_nombre,
    string $curso_nombre
): bool {
    $subject = "📧 Nueva entrega: $actividad_nombre -
$curso_nombre";
    $html = "

                                <div
style='font-family:Arial,sans-serif;max-width:600px;margin:0
auto;'>

                                <div
style='background:linear-gradient(90deg,#2cbaec,#1a8db8);padding:24px 32px;border-radius:12px 12px 0 0;'>

                                <h1
style='color:white;margin:0;font-size:22px;'>D&F
Minspace</h1>
                                </div>
                                <div style='background:#f9fafb;padding:32px;border:1px
solid #e5e7eb;border-top:none;border-radius:0 0 12px 12px;'>
                                <h2
style='color:#1a1a2e;font-size:18px;margin-top:0;'>¡Nueva
entrega recibida!</h2>
                                <p style='color:#555;'>Hola
<strong>$tutor_nombre</strong>,</p>
                                <p style='color:#555;'>El alumno
<strong>$alumno_nombre</strong> ha enviado una entrega que
requiere tu revisión.</p>
                                <div style='background:white;border:1px solid
#e5e7eb;border-radius:8px;padding:18px 22px;margin:20px 0;'>
                                <p><strong>Actividad:</strong>
$actividad_nombre</p>
                                <p><strong>Curso:</strong> $curso_nombre</p>
                                <p><strong>Alumno:</strong> $alumno_nombre</p>
                                </div>
                                <p style='color:#555;'>Ingresa a tu panel para
revisar y calificar la entrega.</p>
```

```

        </div>
    </div>";
    return sendgrid_email($tutor_email, $tutor_nombre,
    $subject, $html);
}

```

Notificación al alumno por calificación

Cuando un tutor asigna una calificación a una entrega previamente realizada, se activa la función `notificar_alumno_calificacion()`. El alumno recibe un correo con su calificación, así como los comentarios que el tutor haya escrito, permitiéndole conocer su desempeño y las áreas de oportunidad detectadas. El mensaje incluye un diseño visual que destaca la puntuación obtenida mediante colores: verde para calificaciones aprobatorias (70% o más) y naranja para calificaciones menores.

Notificación al padre por calificación

La función `notificar_padre_calificacion()` tiene como propósito mantener informados a los padres o tutores legales sobre el progreso académico de sus hijos. Cuando un alumno recibe una calificación, el sistema también notifica al padre vinculado, enviándole un resumen de la actividad y la puntuación obtenida. Esta función es especialmente útil para fomentar la participación de los padres en el proceso de aprendizaje.

Notificación de resumen de entregas pendientes

Adicionalmente, se desarrolló la función `notificar_tutor_resumen_pendientes()`, la cual puede ser ejecutada de forma periódica (mediante un CRON o tarea programada) o bajo demanda. Esta función genera un correo que contiene un listado completo de todas las entregas pendientes de revisión que tiene un tutor, agrupadas por curso y ordenadas por antigüedad. El mensaje incluye una tabla con información de cada entrega (alumno, actividad, fecha de entrega y tiempo transcurrido) y un botón de acceso directo para calificar, lo que facilita la planeación del trabajo del tutor.

9.1.3. Integración con los procesos del sistema

La integración de las notificaciones con los procesos existentes del sistema se realizó de manera no intrusiva, es decir, las llamadas a las funciones de correo se insertaron en los puntos donde ya se ejecutaban las acciones correspondientes, sin modificar la lógica principal de dichos procesos. En el caso de la entrega de una actividad, se agregó la llamada a la función de notificación al tutor dentro del bloque de código que guarda la entrega en la base de datos. De esta manera, cada vez que un alumno completa exitosamente el proceso de entrega, el sistema automáticamente envía el correo correspondiente.

Para la calificación de una actividad, se integró la notificación tanto al alumno como al padre en el archivo `guardar_evaluacion.php`, justo después de que la transacción de base de

datos se ha completado exitosamente. Esto asegura que los correos solo se envían cuando la calificación ha quedado correctamente registrada.

9.1.4. Registro de actividad y monitoreo

Para facilitar la depuración y el monitoreo del sistema de notificaciones, se implementó un mecanismo de registro (logging) que almacena en un archivo de texto (php/sendgrid_log.txt) cada intento de envío de correo. El registro incluye la fecha y hora del intento, el destinatario y el resultado de la operación (éxito o fallo). Esto permite identificar fallos en la comunicación con la API o problemas relacionados con la autenticación.

```
function sg_log($mensaje) {  
    $log_file = __DIR__ . '/sendgrid_log.txt';  
    $timestamp = date('Y-m-d H:i:s');  
    file_put_contents($log_file, "[$timestamp] $mensaje" . PHP_EOL,  
        FILE_APPEND);  
}
```

9.1.5. Seguridad y manejo de credenciales

La API Key de SendGrid se define como constante dentro del archivo de configuración, pero exclusivamente en el entorno local del servidor. Para evitar que estas credenciales sean subidas accidentalmente al repositorio público, se configuró el archivo .gitignore para excluir el archivo php/sendgrid_notificaciones.php del control de versiones. De esta manera, la clave sensible permanece únicamente en el servidor de producción y en los entornos de desarrollo locales, sin riesgo de exposición pública.

9.1.6. Limitaciones consideradas

El plan gratuito de SendGrid utilizado durante el desarrollo permite el envío de hasta 100 correos electrónicos por día, lo cual es suficiente para el volumen esperado del sistema en sus etapas iniciales. Sin embargo, en caso de que el sistema crezca y requiera un mayor número de notificaciones, será necesario migrar a un plan de pago que ofrezca mayores límites de envío. Adicionalmente, el correcto funcionamiento de las notificaciones depende de la conectividad del servidor con la API de SendGrid, por lo que se recomienda monitorear la disponibilidad del servicio.

9.1.7. Resultados obtenidos

Gracias a la integración de la API de SendGrid, el sistema D&F Mindspace cuenta actualmente con un mecanismo de notificaciones automatizado, confiable y fácil de mantener. Los correos electrónicos se entregan de manera oportuna y con un diseño visual agradable, lo que ha mejorado significativamente la comunicación entre los distintos tipos de usuarios de la plataforma. El sistema de registro permite además identificar rápidamente cualquier incidencia relacionada con el envío de correos, facilitando las tareas de mantenimiento y soporte técnico.

10. Seguridad

El sistema D&F Mindspace implementa diversas medidas de seguridad orientadas a proteger la información de los usuarios, así como a garantizar un acceso controlado y confiable a sus funcionalidades.

10.1. Autenticación de usuarios

El sistema D&F Mindspace implementa un mecanismo de autenticación basado en credenciales, mediante el cual los usuarios ingresan su correo electrónico y contraseña a través de la interfaz de inicio de sesión (*login.html*).

Estos datos son enviados al servidor y procesados por el archivo *auth_login.php*, donde se validan contra la información almacenada en la base de datos PostgreSQL. Las contraseñas son gestionadas mediante mecanismos de hash, evitando su almacenamiento en texto plano.

Una vez verificada la información, el sistema crea una sesión utilizando variables de sesión en PHP (*\$_SESSION*), permitiendo mantener al usuario autenticado durante su interacción con el sistema. Además, se asigna un rol al usuario, lo que determina el acceso a las distintas funcionalidades y su redirección al panel correspondiente.

El sistema también incluye un mecanismo de cierre de sesión (*logout.php*) y restringe el acceso a páginas protegidas en caso de no existir una sesión activa.

10.2. Control de acceso por roles

El sistema **D&F Mindspace** implementa un mecanismo de control de acceso basado en roles, el cual permite restringir el acceso a las diferentes funcionalidades según el tipo de usuario autenticado.

Cada usuario cuenta con un rol específico dentro del sistema (administrador, alumno, tutor o padre), el cual es asignado durante el proceso de autenticación y almacenado en las variables de sesión (*\$_SESSION*). Este rol es utilizado para determinar los permisos y las acciones que el usuario puede realizar.

Dependiendo del rol asignado, el sistema redirige al usuario al panel correspondiente (dashboard), donde únicamente se muestran las funcionalidades autorizadas. De igual manera, se restringe el acceso directo a páginas sensibles mediante validaciones que verifican el rol del usuario antes de permitir su ejecución.

Este enfoque garantiza que los usuarios no puedan acceder a información o módulos que no les corresponden, fortaleciendo la seguridad y la integridad del sistema.

10.3. Validación de datos

El sistema **D&F Mindspace** implementa mecanismos de validación de datos con el objetivo de garantizar la integridad y consistencia de la información ingresada por los usuarios.

Estas validaciones se realizan principalmente en los formularios del sistema, verificando que los campos requeridos sean completados correctamente antes de ser enviados al servidor. Asimismo, se aplican controles básicos como la verificación de formatos y la prevención de datos vacíos o inválidos.

La validación contribuye a reducir errores durante el procesamiento de la información y evita el ingreso de datos inconsistentes en la base de datos. Además, permite mejorar la experiencia del usuario al proporcionar retroalimentación inmediata en caso de errores en el llenado de formularios.

10.4. Manejo de sesiones

El sistema **D&F Mindspace** utiliza mecanismos de manejo de sesiones mediante PHP para controlar el estado de autenticación de los usuarios durante su interacción con la aplicación.

Una vez que el usuario ha iniciado sesión correctamente, se crean variables de sesión (***\$_SESSION***) que almacenan información relevante como el identificador del usuario y su rol. Estas variables permiten mantener la sesión activa y gestionar el acceso a las diferentes funcionalidades del sistema.

El sistema verifica la existencia de una sesión activa antes de permitir el acceso a páginas protegidas. En caso de que no exista una sesión válida, el usuario es redirigido automáticamente a la página de inicio de sesión, evitando accesos no autorizados.

Asimismo, se implementa un mecanismo de cierre de sesión mediante el archivo ***logout.php***, el cual destruye las variables de sesión activas, garantizando que el usuario cierre correctamente su sesión y previniendo accesos indebidos en equipos compartidos.

10.5. Seguridad en la base de datos

El sistema D&F Mindspace gestiona el acceso a la base de datos PostgreSQL mediante un archivo de configuración (*config.php*), en el cual se definen los parámetros necesarios para establecer la conexión, tales como el host, puerto, nombre de la base de datos, usuario y contraseña.

El acceso a la base de datos se realiza desde el backend desarrollado en PHP, permitiendo la ejecución de consultas para la gestión de la información del sistema. Estas consultas permiten realizar operaciones de inserción, consulta, actualización y eliminación de datos.

Asimismo, el archivo de configuración debe mantenerse protegido y fuera del acceso público, evitando que usuarios no autorizados puedan visualizar las credenciales de conexión. Esto es fundamental para garantizar la confidencialidad y seguridad de la información almacenada.

10.6. Manejo de errores

El sistema **D&F Mindspace** implementa un manejo básico de errores con el objetivo de identificar y facilitar la detección de fallas durante la ejecución de la aplicación.

Actualmente, los errores generados en el sistema son mostrados directamente en pantalla, lo que permite a los desarrolladores identificar problemas de manera rápida durante la fase de desarrollo y pruebas. Esto resulta útil para depuración y corrección de fallos en el código.

Sin embargo, la exposición directa de errores puede representar un riesgo de seguridad en entornos de producción, ya que podría revelar información sensible del sistema, como estructuras internas, rutas de archivos o detalles de la base de datos.

Por esta razón, se recomienda implementar un sistema de manejo de errores más robusto, que incluya el registro de errores en archivos de log y la ocultación de mensajes técnicos al usuario final. De esta manera, se mejora la seguridad del sistema sin perder la capacidad de monitoreo y diagnóstico.

10.8. Respaldos

El sistema **D&F Mindspace** contempla mecanismos de respaldo orientados a garantizar la disponibilidad y recuperación de la información en caso de fallos, pérdidas de datos o caídas del sistema.

Actualmente, los respaldos pueden realizarse mediante copias de seguridad de la base de datos PostgreSQL, las cuales permiten restaurar la información en caso de incidentes. Estas copias pueden ser generadas de forma manual utilizando herramientas propias del gestor de base de datos.

Adicionalmente, como parte de la mejora en la infraestructura del sistema, se contempla la implementación de un esquema de alta disponibilidad basado en replicación de base de datos, tomando como referencia las prácticas realizadas durante el desarrollo del proyecto.

En este enfoque, se utiliza un servidor principal (master) y un servidor espejo (standby), el cual mantiene una copia sincronizada de la base de datos mediante el mecanismo de **replicación física (Hot Standby)**. Este proceso permite que los cambios realizados en el servidor principal sean replicados automáticamente en el servidor secundario en tiempo real, garantizando la consistencia de la información.

La replicación se realiza mediante la transferencia continua de archivos WAL (Write-Ahead Logging), los cuales contienen los registros de transacciones del sistema. Estos archivos son enviados desde el servidor principal hacia el servidor espejo, permitiendo mantener una sincronización constante entre ambos nodos.

El servidor espejo opera en modo de solo lectura, lo que asegura la integridad de los datos y permite su utilización como respaldo activo en caso de fallo del servidor principal. En este escenario, el servidor secundario puede ser promovido para asumir el rol principal, garantizando la continuidad del servicio.

Este enfoque de respaldo no solo permite la recuperación de información, sino que también proporciona tolerancia a fallos y mejora la disponibilidad del sistema, siendo una estrategia robusta para entornos que requieren alta confiabilidad.

11. Errores

El sistema D&F Mindspace implementa un mecanismo estructurado de manejo de errores, el cual permite tanto la detección de fallos durante la ejecución como su registro para análisis y mantenimiento posterior.

El manejo de errores se realiza principalmente mediante el uso de estructuras de control como bloques try-catch en PHP, lo que permite capturar excepciones generadas durante la ejecución del sistema. Este enfoque es especialmente utilizado en operaciones críticas, como la interacción con la base de datos, donde pueden presentarse fallos de conexión, errores en consultas o inconsistencias en los datos. Gracias a este mecanismo, el sistema puede controlar los errores sin interrumpir completamente su funcionamiento, permitiendo gestionar las fallas de manera más segura y ordenada.

Adicionalmente, el sistema cuenta con un mecanismo de registro de errores (logging), en el cual los eventos de fallo son almacenados dentro de la base de datos. Este registro no está orientado al usuario final, sino al administrador del sistema, quien puede acceder a esta información mediante herramientas internas para monitorear el comportamiento de la aplicación. Esto permite llevar un control histórico de errores, identificar patrones de fallos y facilitar las tareas de diagnóstico y corrección.

Durante la fase de desarrollo, el sistema permite la visualización de errores directamente en pantalla, lo cual facilita la detección inmediata de problemas y acelera el proceso de depuración. Esta funcionalidad es soportada por la configuración del entorno de desarrollo (XAMPP), que habilita la muestra de errores de PHP y del servidor web.

Sin embargo, en entornos de producción se recomienda restringir la visualización de estos mensajes, ya que podrían exponer información sensible del sistema, como rutas internas, estructura del código o detalles de la base de datos. En su lugar, los errores deben ser gestionados de forma interna mediante el sistema de logs.

En conjunto, el sistema combina la captura controlada de excepciones con el registro estructurado de errores, lo que contribuye a mejorar la estabilidad, seguridad y mantenibilidad de la aplicación.

12. Respaldos

El sistema **D&F Mindspace** implementa estrategias de respaldo orientadas a garantizar la disponibilidad, integridad y recuperación de la información ante posibles fallos del sistema.

Tipos de respaldo

El sistema contempla dos tipos de respaldo:

- **Respaldo en la nube:**
Consiste en el almacenamiento de copias de seguridad en un servicio externo como **Google Drive**, lo que permite disponer de la información de forma remota. Sin embargo, este método puede resultar más lento y menos eficiente para la restauración inmediata del sistema, por lo que se utiliza principalmente como un respaldo complementario.
- **Respaldo por servidor completo (replicación):**
Este es el mecanismo principal de respaldo del sistema, basado en un esquema de alta disponibilidad mediante replicación de base de datos.

12.1. Respaldo por servidor completo

El sistema implementa un modelo de replicación utilizando un servidor principal (master) y un servidor espejo (standby), mediante la técnica de **replicación física (Hot Standby)** en PostgreSQL.

En este esquema, se realiza una clonación inicial del servidor principal utilizando herramientas como ***pg_basebackup***, lo que permite copiar completamente el clúster de la base de datos, incluyendo datos, configuraciones y archivos necesarios para su funcionamiento.

Posteriormente, el servidor espejo se mantiene sincronizado en tiempo real mediante el envío continuo de registros WAL (Write-Ahead Logging), los cuales contienen todas las transacciones realizadas en el servidor principal. Estos registros son aplicados automáticamente en el servidor secundario, garantizando que ambos nodos mantengan la misma información de manera consistente.

El servidor espejo opera en modo **Hot Standby**, lo que significa que se mantiene en estado de solo lectura, permitiendo consultas sin afectar la integridad de la replicación. Además, este servidor puede ser promovido a servidor principal en caso de fallo del nodo original, asegurando la continuidad del sistema.

Durante la implementación de este esquema, se configuraron archivos esenciales como *pg_hba.conf* y *postgresql.conf*, los cuales permiten definir reglas de autenticación, parámetros de conexión y opciones de replicación necesarias para el correcto funcionamiento del sistema.

Asimismo, se realizaron verificaciones mediante herramientas internas de PostgreSQL, como *pg_stat_replication*, permitiendo confirmar que la sincronización entre ambos servidores se mantiene activa, estable y en tiempo real.

12.2. Ubicación de los respaldos

El sistema D&F Mindspace cuenta con una estrategia de almacenamiento de respaldos distribuida en dos niveles, lo que permite aumentar la seguridad y disponibilidad de la información.

El respaldo principal se encuentra en el servidor espejo (standby), el cual forma parte de la infraestructura interna del sistema. Este servidor mantiene una copia sincronizada en tiempo real de la base de datos mediante replicación, lo que garantiza que ante cualquier fallo del servidor principal, la información pueda recuperarse de forma inmediata sin pérdida significativa de datos. Al estar dentro del mismo entorno controlado, este tipo de respaldo permite una recuperación rápida y eficiente del sistema.

Por otro lado, se dispone de un respaldo complementario en la nube mediante Google Drive, el cual funciona como almacenamiento externo. Este tipo de respaldo proporciona una capa adicional de seguridad, ya que permite conservar copias de la información fuera del entorno local, protegiendo los datos ante fallos críticos como daños en hardware, errores graves del sistema o pérdida total de la infraestructura principal.

La combinación de ambas ubicaciones permite contar con una estrategia de respaldo robusta, equilibrando la rapidez de recuperación (servidor espejo) con la seguridad ante desastres (almacenamiento en la nube).

12.3. Periodicidad

La periodicidad de los respaldos en el sistema D&F Mindspace varía según el tipo de mecanismo implementado, garantizando tanto actualización constante como almacenamiento seguro a largo plazo.

En el caso del respaldo por servidor completo (replicación), la periodicidad es continua y en tiempo real. Esto se debe a que la sincronización entre el servidor principal y el servidor espejo se realiza mediante la transmisión constante de registros WAL, lo que permite que cada cambio en la base de datos sea replicado automáticamente en el servidor secundario sin necesidad de intervención manual. Este enfoque asegura que el sistema cuente en todo momento con una copia actualizada de la información.

En cuanto al respaldo en la nube, la periodicidad es de tipo programada o manual, realizándose en intervalos definidos según las necesidades del sistema, como puede ser semanalmente o antes de realizar cambios importantes. Este tipo de respaldo no es continuo debido al tiempo que implica la transferencia de datos hacia un servicio externo, por lo que se utiliza principalmente como medida preventiva adicional.

De esta manera, el sistema combina un respaldo inmediato y constante con uno externo y periódico, logrando un equilibrio entre disponibilidad, rendimiento y seguridad de la información.

13. Mantenimiento

El monitoreo del sistema **D&F Mindspace** permite supervisar de manera continua el estado general de la aplicación, con el objetivo de detectar posibles fallas, prevenir errores críticos y garantizar un funcionamiento estable.

Para ello, se llevan a cabo diversas actividades que permiten evaluar tanto el comportamiento del servidor como la integridad de los datos y el rendimiento de los módulos del sistema.

Entre las principales actividades de monitoreo se encuentran:

- **Verificación del funcionamiento del servidor Apache:**

Se supervisa que el servidor web Apache se encuentre en ejecución y respondiendo correctamente a las solicitudes de los usuarios. Esto incluye la validación de servicios activos, puertos habilitados y tiempos de respuesta, con el fin de asegurar la disponibilidad del sistema.

- **Supervisión de la conexión con la base de datos PostgreSQL:**

Se verifica de forma periódica que la conexión entre la aplicación y la base de datos sea estable, evitando fallos en la ejecución de consultas. Esto incluye la validación de credenciales, disponibilidad del servidor de base de datos y correcta comunicación con la máquina virtual donde se encuentra alojado.

- **Revisión de logs de errores almacenados en la base de datos:**

El sistema cuenta con un registro de errores que permite identificar fallos ocurridos durante la ejecución. Estos logs son analizados por el administrador para detectar patrones, identificar módulos con problemas recurrentes y tomar acciones correctivas de manera oportuna.

- **Evaluación del rendimiento en los diferentes módulos del sistema:**

Se analiza el comportamiento de los distintos módulos (usuarios, cursos, actividades, dashboards, etc.) para detectar posibles lentitudes, fallos en la carga de información o problemas en la ejecución de procesos. Esto permite optimizar el sistema y mejorar la experiencia del usuario.

Adicionalmente, el monitoreo permite anticipar posibles fallas antes de que afecten directamente al usuario final, facilitando la toma de decisiones para el mantenimiento preventivo del sistema.

13.1. Respaldo de la información

El sistema D&F Mindspace implementa un esquema de respaldo de la información orientado a garantizar la disponibilidad, integridad y recuperación de los datos ante posibles fallos del sistema, errores humanos o incidentes en la infraestructura.

El mecanismo principal de respaldo se basa en un modelo de alta disponibilidad mediante replicación de base de datos, utilizando un servidor principal (master) y un servidor espejo (standby). Este enfoque permite mantener una copia completa y actualizada de la base de datos en todo momento.

El proceso inicia con una copia base del servidor principal, la cual incluye todos los datos y configuraciones necesarias para el funcionamiento del sistema. Posteriormente, se establece un mecanismo de sincronización continua mediante la transmisión de registros WAL (Write-Ahead Logging), los cuales contienen todas las transacciones realizadas en el sistema. Estos registros son enviados al servidor espejo, donde son aplicados automáticamente, garantizando la consistencia de la información en ambos nodos.

Este tipo de respaldo permite que, en caso de fallo del servidor principal, el servidor secundario pueda asumir su función mediante un proceso de promoción, reduciendo significativamente el tiempo de inactividad y evitando la pérdida de datos.

Adicionalmente, el sistema contempla un respaldo complementario en la nube mediante Google Drive, el cual permite almacenar copias externas de la información. Este tipo de respaldo proporciona una capa adicional de seguridad ante fallos críticos, como daños físicos en el servidor o pérdida total de la infraestructura local. Sin embargo, debido al tiempo requerido para la transferencia de datos, este mecanismo no se utiliza como respaldo principal.

El esquema implementado permite combinar un respaldo en tiempo real (replicación) con un respaldo externo (nube), logrando un equilibrio entre rapidez de recuperación y protección ante desastres.

Asimismo, se recomienda realizar verificaciones periódicas del estado de la replicación y de la integridad de los respaldos almacenados, con el fin de asegurar que el sistema de respaldo se mantenga operativo y confiable en todo momento.

13.2. Restauración del sistema

El sistema D&F Mindspace cuenta con procedimientos de restauración orientados a recuperar la operación del sistema ante fallos, pérdida de información o interrupciones del servicio.

El principal mecanismo de restauración se basa en el uso del servidor espejo (standby) dentro del esquema de replicación implementado. En caso de fallo del servidor principal, el servidor secundario puede ser promovido a servidor principal, permitiendo restablecer el servicio de manera rápida y con una mínima pérdida de datos. Este proceso garantiza la continuidad operativa del sistema, ya que la base de datos en el servidor espejo se mantiene sincronizada en tiempo real.

El procedimiento general de restauración mediante replicación consiste en:

- 1. Detectar la falla o indisponibilidad del servidor principal.***
- 2. Verificar el estado del servidor espejo y su nivel de sincronización.***
- 3. Promover el servidor secundario a servidor principal mediante herramientas de PostgreSQL.***
- 4. Redirigir las conexiones del sistema hacia el nuevo servidor activo.***

Este proceso permite reducir significativamente el tiempo de inactividad y asegura que los usuarios puedan continuar utilizando el sistema sin interrupciones prolongadas.

Adicionalmente, el sistema contempla la restauración mediante respaldos externos almacenados en Google Drive, los cuales pueden ser utilizados en escenarios críticos donde ambos servidores (principal y espejo) presenten fallos o pérdida de información. En estos casos, se realiza la recuperación mediante la importación de la copia de seguridad en un nuevo entorno de base de datos.

Asimismo, en caso de corrupción o pérdida de archivos del sistema, se pueden restaurar los componentes de la aplicación reemplazando los archivos desde una versión funcional previamente respaldada o desde el repositorio del sistema.

Se recomienda realizar pruebas periódicas de restauración para asegurar que los procedimientos definidos sean efectivos y que los respaldos disponibles puedan ser utilizados correctamente en situaciones reales.

13.3. Mantenimiento preventivo y correctivo

El sistema D&F Mindspace implementa estrategias de mantenimiento preventivo y correctivo con el objetivo de garantizar su estabilidad, seguridad y correcto funcionamiento a lo largo del tiempo.

El mantenimiento preventivo consiste en la realización de actividades periódicas destinadas a evitar fallos antes de que ocurran. Estas acciones incluyen la revisión constante del estado del servidor Apache, la verificación de la conectividad con la base de datos PostgreSQL y el monitoreo del rendimiento general del sistema. Asimismo, se lleva a cabo la revisión de los registros de errores almacenados en la base de datos, con el fin de identificar posibles incidencias recurrentes y anticipar problemas futuros.

Dentro del mantenimiento preventivo también se consideran tareas como la validación del correcto funcionamiento de los mecanismos de respaldo y replicación, asegurando que el servidor espejo se mantenga sincronizado con el servidor principal. De igual manera, se realizan pruebas periódicas de los módulos del sistema (usuarios, cursos, actividades, dashboards, entre otros), con el objetivo de detectar posibles fallas de funcionamiento o degradación en el rendimiento.

Por otro lado, el mantenimiento correctivo se enfoca en la detección y solución de errores que ya se han presentado durante la operación del sistema. Este proceso se apoya principalmente en el análisis de los logs de errores, los cuales permiten identificar la causa de los fallos y aplicar las correcciones necesarias.

Las acciones de mantenimiento correctivo pueden incluir la corrección de errores en el código fuente, ajustes en consultas a la base de datos, optimización de procesos, así como la actualización o reemplazo de componentes del sistema que presenten fallos. En caso de incidentes mayores, también se pueden aplicar procedimientos de restauración utilizando los mecanismos de respaldo disponibles.

La combinación de mantenimiento preventivo y correctivo permite no solo solucionar problemas existentes, sino también reducir la probabilidad de fallos futuros, asegurando la continuidad operativa y la mejora constante del sistema.

14. Contacto

Para cualquier consulta técnica, reporte de errores o solicitud de soporte relacionada con el sistema **D&F Mindspace**, se dispone de los siguientes medios de contacto:

- **Responsables del desarrollo:**
Delgado Ramírez María Fernanda
López Cervantes Derek Alejandro
- **Correo electrónico:**
mariferdelgadoim@gmail.com
lopezcderek1720@gmail.com
- **Número de Teléfono:**
656-576-4750
657-143-4968
- **Institución:**
Tecnológico Nacional de México: Instituto Tecnológico de Ciudad Juárez
- **Área responsable:**
Desarrollo y soporte técnico

Adicionalmente, el código fuente del sistema se encuentra gestionado mediante la plataforma GitHub, lo que permite el seguimiento de cambios, control de versiones y colaboración entre los desarrolladores.

El tiempo de respuesta a solicitudes de soporte puede variar dependiendo de la disponibilidad de los desarrolladores.